

Software Architecture and Design Specification (SAD)

Project: Railway Reservation System (RRS)
Version: 1.0
Authors: Team 2 (Praveen – PES2UG24CS373, Praveen Balaji P – PES2UG24CS374, Prithvi S Paraddi – PES2UG24CS375, Priyaan G – PES2UG24CS376)
Date: 04-10-2026
Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team 2 (Praveen, Praveen Balaji P, Prithvi S Paraddi, Priyaan G)	Initial complete SAD specification with UML Component Diagram, 2 Sequence Diagrams, Layered Architectural Pattern Rationale, STRIDE Threat Model, C++ Interface APIs, and ADRs.

Approvals

Role	Name	Signature / Date
Author		
Course Coordinator / Faculty Evaluator		

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Railway Reservation System (RRS) v1.0, developed by Team 2. It defines the structural decomposition, subsystem interfaces, dynamic operational behavior, architectural patterns, STRIDE threat models, and low-level C/C++ API contracts. It bridges the conceptual requirements defined in the SRS and the verification plans in the STP with a concrete, thread-safe engineering design.

1.2 Scope

The scope covers the end-to-end architecture of the terminal and client-server railway reservation platform, including:

- Terminal presentation layer with ANSI-formatted CLI/TUI and non-canonical password masking.
- Session security, salted SHA-256 cryptographic credential validation, and Role-Based Access Control (RBAC).
- Real-time train route discovery, timetable graph traversal, and multi-class berth inventory tracking (1A, 2A, 3A, SL, CC).
- Multi-passenger reservation (1-6 seats), dynamic age-concession pricing, berth allocation, and unique 10-digit PNR generation.
- Atomic transaction management with POSIX mutual exclusion and record-level file locking ('fcntl') preventing double-booking.
- Cancellation refund deductions and automated First-In, First-Out (FIFO) queue cascades (RAC to CNF, WL to RAC).
- Administrative master catalog onboarding, quota configuration, passenger manifest charting, and rolling SHA-256 audit logging.
- Physical railway interlocking systems, track signaling telemetry, and commercial banking clearinghouse infrastructure are excluded.

1.3 Audience

Developers, QA engineers, security auditors, course instructors, evaluators, and future system maintenance teams.

1.4 Definitions

- **RRS**: Railway Reservation System.
- **PNR**: Passenger Name Record (Unique 10-digit reservation identifier with embedded CRC-32 checksum).
- **CNF / RAC / WL**: Confirmed Berth / Reservation Against Cancellation / Waiting List.
- **ADR**: Architectural Decision Record.
- **RBAC**: Role-Based Access Control (Passenger, Clerk, Administrator).
- **STRIDE**: Security threat categorization model (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).
- **RAII**: Resource Acquisition Is Initialization (Modern C++ memory management idiom).
- **POSIX**: Portable Operating System Interface standards (threading, file locks, system calls).

2. Document Overview

2.1 How to use this document

This document provides architectural and design deliverables organized into two primary tracks:

1. Architectural Track (Section 3): System-level decomposition, UML Component Diagram, architectural style rationale, technology stack specifications, requirement traceability, and STRIDE security threat modeling.
2. Design Track (Section 4): Detailed component interactions, two UML Sequence Diagrams (Ticket Booking and Cancellation Flows), formal C/C++ API interface definitions, error handling strategies, UX design guidelines, and Architectural Decision Records (ADRs).

2.2 Related Documents

- **Baseline SRS:** Software Requirements Specification (SRS) for Railway Reservation System v1.0 (Team 2).
- **Verification STP:** Software Test Plan (STP) for Railway Reservation System v1.0 (DEGOL100400577, Team 2).
- **Traceability Matrix (RTM):** Bidirectional Traceability Matrix mapping SRS requirements to test cases.
- **Standards Reference: IEEE 42010:2011:** Systems and Software Engineering — Architecture Description.

3. Architecture

3.1 Goals & Constraints

Architectural Goals:

- **High Performance:** Train search queries executed in ≤ 150 ms across 500+ trains; booking commits in ≤ 500 ms.
- **Concurrency & Race-Free Operation:** Support at least 50 simultaneous counter clerk terminal sessions without deadlock or double-allocated berths.
- **Data Durability:** 99.95% transaction durability with zero database corruption upon unexpected process termination (`SIGKILL`).
- **Memory Safety & Robustness:** Strict adherence to ISO C++17 memory management (RAII), passing AddressSanitizer and Valgrind audits with 0 leaks.

Architectural Constraints:

- Core implementation strictly constrained to ISO C++17 / C standard library.
- Reliance on POSIX system primitives (`pthread`, `fcntl`) for multi-threading and record locking.
- Terminal rendering restricted to ANSI VT100/Xterm text interface with non-canonical password masking.
- Persistent storage implemented as compact, deterministic fixed-length binary records with write-ahead logging (WAL).

3.2 Stakeholders & Concerns

- **Passengers (Customers):** Low latency, transparent berth availability, accurate fare breakdowns, instant PNR issuance, data privacy.
- **Booking Clerks:** High data entry throughput, keyboard shortcuts, immediate ticket receipt printing, daily cash ledger reconciliation.
- **Railway Administrators:** Master catalog controls, coach capacity adjustments, chart manifest locking, immutable audit log tamper-proofing.
- **Developers & QA:** Modularity, separation of concerns, clean ISO C++17 compilation under `-Wall -Wextra`, comprehensive unit testability.
- **Course Evaluators:** Rigorous compliance with software engineering templates, IEEE 42010 standards, and full SRS/STP traceability.

3.3 Component (UML) Diagram

Figure 3.1 illustrates the structural component architecture of the Railway Reservation System. The system is organized into three major architectural tiers: the Presentation & Session Layer, the Core Domain & Application Service Layer, and the Persistence, Audit & Storage Layer.

Figure 3.1: UML Component Diagram — Railway Reservation System (RRS)
Architectural Subsystems, Modules, and Interface Boundaries (Project DEGOL100400577 | Team 2)

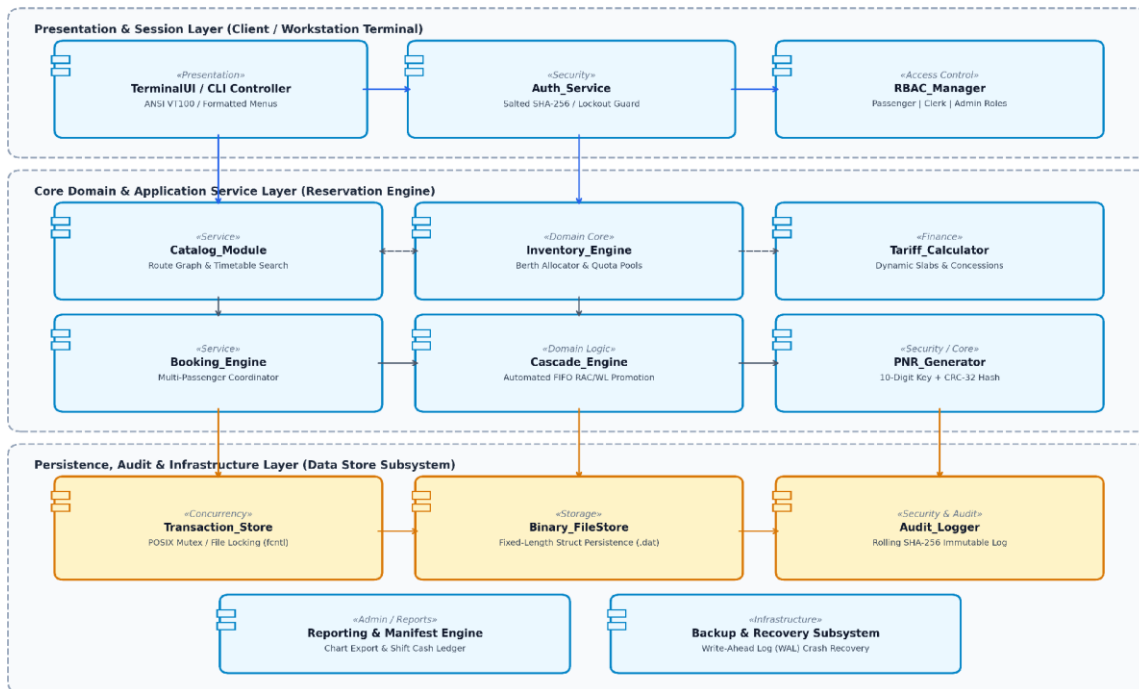


Figure 3.1: UML Component Diagram — Railway Reservation System Architectural Subsystems

3.4 Component Descriptions

- **TerminalUI / CLI Controller:** Manages console menus, captures keystrokes, masks password inputs (`\*`), and formats tabular ASCII outputs.
- **Auth_Service:** Validates user credentials against salted SHA-256 digests and enforces 3-attempt account lockout policies.
- **RBAC_Manager:** Enforces Role-Based Access Control, mapping authenticated sessions to Passenger, Clerk, or Administrator permissions.
- **Catalog_Module:** Maintains in-memory station graphs and timetables, executing multi-station route searches and distance computations.
- **Inventory_Engine & BerthAllocator:** Tracks live multi-class seat quotas (1A, 2A, 3A, SL, CC), evaluates quota availability, and allocates physical berths.
- **Tariff_Calculator:** Computes dynamic distance-based fares, applies senior citizen (40%) and child (50%) concessions, and calculates cancellation refund slabs.
- **Booking_Engine:** Coordinates multi-passenger booking forms (1-6 seats), generates unique 10-digit PNRs with CRC-32 checksums, and orchestrates atomic commits.
- **Cascade_Engine:** Executes automated First-In First-Out (FIFO) queue promotions, transitioning RAC to CNF and WL to RAC upon seat cancellation.
- **Transaction_Store:** Enforces thread safety and atomic file serialization using POSIX mutexes and file record locking (`fcntl`).
- **Binary_FileStore:** Manages binary persistence for `trains.dat`, `passengers.dat`, and `pnr\_records.dat` with fixed-stride record offsets.
- **Audit_Logger:** Records synchronous, immutable append-only logs for all booking, cancellation, and admin operations using rolling SHA-256 block checksums.
- **Reporting & Manifest Engine:** Generates final passenger manifest charts 4 hours prior to departure and reconciles counter cash collections.

3.5 Chosen Architecture Pattern and Rationale

Pattern: Layered (Tiered) Architecture with Event-Driven Queue Cascading.

Rationale:

1. Separation of Concerns: Clear isolation between Presentation (TerminalUI), Business Services (BookingEngine, CatalogModule), Domain Logic (BerthAllocator, CascadeEngine), and Data Storage (FileStore, TransactionStore).
2. High Performance & Low Latency: Avoids unnecessary inter-process network hops and JSON serialization overhead, allowing in-memory pointer dereferencing and microsecond-level berth state evaluation.
3. Concurrency Safety: Allows fine-grained mutex locking at the specific train and coach inventory level rather than locking the entire system.

Alternatives Considered & Rejected:

- Microservices Architecture: Rejected due to prohibitive operational complexity, network latency, distributed transaction overhead (2PC/Saga), and excessive resource consumption for an embedded/workstation C++ railway appliance.
- Monolithic Procedural Design: Rejected due to tight coupling, poor testability, and high risk of race conditions during multi-terminal expansion.

3.6 Technology Stack & Data Stores

- **Core Language & Runtime:** ISO C++17 (compiled with GCC 11+ / Clang 14+ with `-std=c++17 -Wall -Wextra -Wpedantic`).
- **Concurrency & OS APIs:** POSIX Threads (`<pthread.h>` / `<mutex>`), POSIX File Locks (`<fcntl.h>`), ANSI VT100 / POSIX Termios (`<termios.h>`).
- **Cryptographic Engine:** Custom cryptographic module executing salted SHA-256 digests and 32-bit CRC checksum generation.
- **Data Stores & Serialization:** Compact binary files (`.dat`) with fixed-length C++ structs (`TrainRecord`, `PNRRecord`, `PassengerRecord`), enabling $O(1)$ random access via `fseek`.
- **Durability & Audit:** Synchronous write-ahead log (`wal.log`) ensuring atomic crash recovery, and append-only tamper-evident audit file (`audit.log`).

3.7 Risks & Mitigations

Architectural Risk	Impact & Likelihood	Engineering Mitigation
Double-booking of berths under simultaneous booking clerk requests.	Critical / Medium	Enforce fine-grained in-memory train mutexes combined with POSIX write-exclusive record locks (<code>F_WRLCK</code>) during allocation.
Memory leaks or buffer overflow crashes in C/C++ string manipulations.	Critical / Low	Strictly enforce modern C++ RAI (<code>std::unique_ptr</code> , <code>std::string</code>), prohibit unsafe functions (<code>gets</code> , <code>strcpy</code>), and validate via AddressSanitizer.
File store corruption during power failure or unexpected process termination.	High / Low	Implement two-phase atomic write-ahead logging (WAL) and write-to-temporary with atomic filesystem rename (<code>rename()</code>).
Starvation or race conditions in waitlist queue promotions during cancellations.	Medium / Low	Encapsulate queue operations within an atomic transaction block under a dedicated cascade lock, executing strict FIFO order.

3.8 Traceability to Requirements

SRS Requirement Category	SRS Req IDs	Architectural Component Mapping
Authentication & RBAC	RRS-F-001, RRS-F-002, RRS-F-003, RRS-SR-001, RRS-SR-002	TerminalUI, Auth_Service, RBAC_Manager, UserStore
Train Search & Inquiries	RRS-F-004, RRS-F-005, RRS-F-006, RRS-NF-001	Catalog_Module, Inventory_Engine
Booking, Tariff & PNR	RRS-F-007 to RRS-F-013, RRS-NF-002, RRS-NF-004, RRS-SR-005	Booking_Engine, BerthAllocator, Tariff_Calculator, PNR_Generator, Transaction_Store
Cancellation & Queue Cascade	RRS-F-014, RRS-F-015, RRS-F-016	Booking_Engine, Tariff_Calculator, Cascade_Engine, Inventory_Engine
Admin Master & Charting	RRS-F-017 to RRS-F-021	Catalog_Module, ReportingModule, LedgerModule
Durability, Audit & Security	RRS-NF-003, RRS-NF-005, RRS-SR-003, RRS-SR-004	Transaction_Store, Binary_FileStore, Audit_Logger, InputSanitizer

3.9 Security Architecture (STRIDE Threat Model)

STRIDE Category	Identified Threat in RRS	Security Objective	Architectural Mitigation Mechanism
Spoofing	Adversary impersonating booking clerk or administrator.	Authentication	Salted SHA-256 password hashing; masked terminal input; session token verification.
Tampering	Modifying binary ticket records or counterfeiting PNRs.	Integrity	32-bit CRC verification checksum embedded in PNR; POSIX file permissions (0600); WAL checksums.
Repudiation	Operator denying processing a cancellation or booking.	Non-Repudiation	Immutable append-only audit log with rolling SHA-256 block hashes, timestamp, and User ID.
Information Disclosure	Snooping passenger contact details or plaintext credentials.	Confidentiality	Memory scrubbing after authentication; passwords masked on terminal ('*'); restricted file stores.
Denial of Service (DoS)	Flooding booking terminals or exhaustively locking berths.	Availability	Account lockout after 3 failed attempts; bounded transaction queues; thread lock timeouts.
Elevation of Privilege	Passenger accessing admin master route or tariff menus.	Authorization	Strict Role-Based Access Control (RBAC); compile-time menu dispatch verification.

4. Design

4.1 Design Overview

The low-level design structures the C++ codebase into cohesive, loosely-coupled classes organized into dedicated namespaces. State transitions across the reservation lifecycle are strictly deterministic: from UNRESERVED to CONFIRMED, RAC, or WAITING_LIST, and from ACTIVE to CANCELLED. Concurrency is orchestrated via coarse-grained reader locks for inquiries and fine-grained writer mutexes for berth allocation and cancellation cascades.

4.2 UML Sequence Diagrams

In accordance with software engineering design standards, this section presents two detailed UML Sequence Diagrams modeling the dynamic execution flows for the system's core transactional operations.

4.2.1 Sequence Diagram 1: Multi-Passenger Ticket Booking & PNR Generation Flow

Figure 4.1 details the multi-step transaction flow when a user or counter clerk books a ticket. It models authentication verification, seat availability checking, acquiring the train's berth mutex lock, physical berth assignment, dynamic fare calculation, 10-digit PNR generation with CRC-32 checksum, atomic commit to persistent storage with write-ahead logging, and synchronous audit logging.

Figure 4.1: UML Sequence Diagram — Multi-Passenger Ticket Booking & PNR Generation Flow
End-to-End Transaction Coordination with Mutex Lock (Project DEGOL100400577 | Team 2)

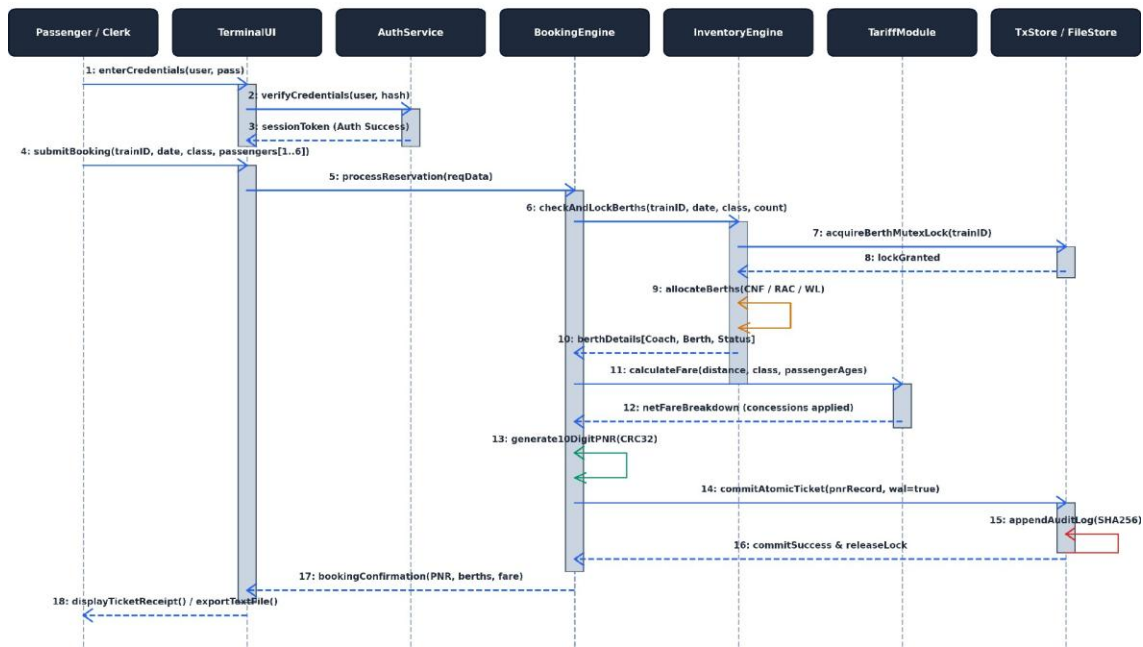


Figure 4.1: UML Sequence Diagram — Multi-Passenger Ticket Booking & PNR Generation Flow

4.2.2 Sequence Diagram 2: Ticket Cancellation & Automated FIFO Queue Cascade Flow

Figure 4.2 illustrates the cancellation lifecycle. When a cancellation request is submitted, the system retrieves the active PNR record, computes deduction penalties based on departure departure time slabs, acquires the train's cascade mutex lock, restores seat inventory, and executes an atomic FIFO promotion: upgrading the highest-priority RAC passenger to CNF and promoting the first WL passenger to RAC. The transaction logs a rolling SHA-256 audit record and returns a refund voucher to the user.

Figure 4.2: UML Sequence Diagram — Ticket Cancellation, Refund & Automated FIFO Queue Cascade
Berth Reallocation & Waitlist Promotion Flow (Project DEGOL100400577 | Team 2)

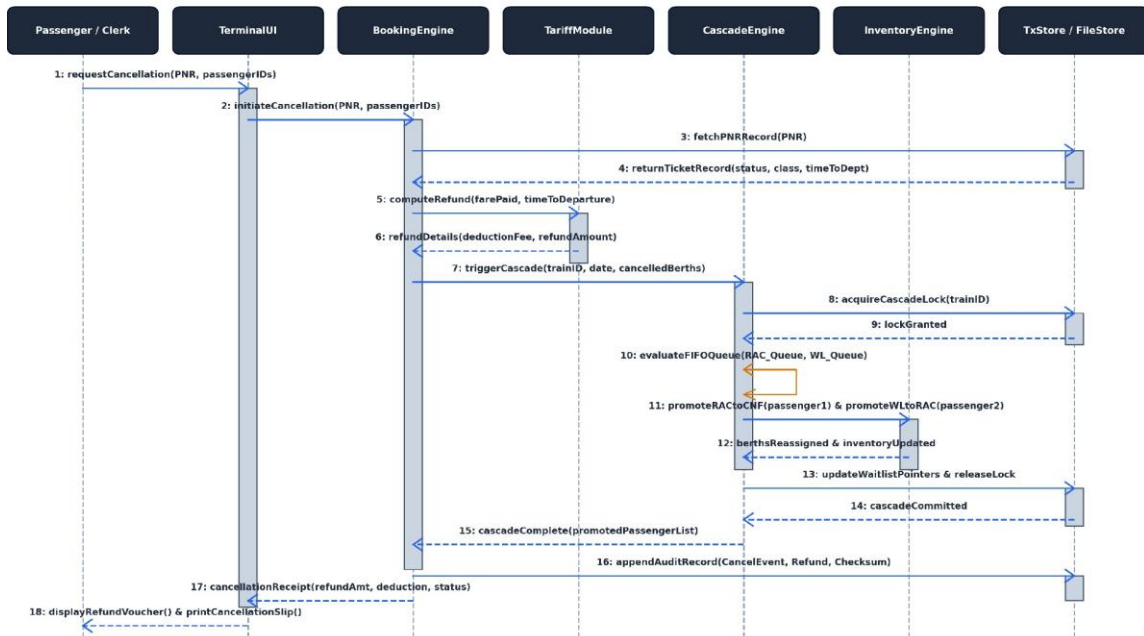


Figure 4.2: UML Sequence Diagram — Ticket Cancellation, Refund & Automated FIFO Queue Cascade

4.3 API Design (Interface Definitions)

This section specifies the programmatic interfaces for the core architectural components in modern C++17 syntax.

Component 1: BookingEngine Interface ('IBookingEngine.hpp')

```

```cpp
namespace RRS::Core {
 struct PassengerInput {
 std::string name;
 uint8_t age;
 char gender; // 'M', 'F', 'O'
 BerthPreference preference; // LOWER, MIDDLE, UPPER, SIDE_LOWER, NONE
 };

 struct BookingRequest {
 std::string sessionToken;
 uint32_t trainID;
 std::string journeyDate; // YYYY-MM-DD
 CoachClass travelClass; // AC_1A, AC_2A, AC_3A, SLEEPER, CHAIR_CAR
 QuotaType quota; // GENERAL, TATKAL, LADIES
 std::vector<PassengerInput> passengers; // 1 to 6
 };

 struct BookingResponse {
 StatusCode status; // SUCCESS, ERR_SEATS_EXHAUSTED, ERR_INVALID_DATA
 std::string pnrNumber; // 10-digit unique key
 double totalFare;
 std::vector<AllocatedBerth> allocatedBerths;
 std::string bookingTimestamp;
 };

 class IBookingEngine {
 public:

```

```

 virtual ~IBookingEngine() = default;
 virtual BookingResponse bookTicket(const BookingRequest& request) = 0;
 virtual CancellationResponse cancelTicket(const std::string& pnr, const std::vector<uint32_t>& passengerIDs) =
0;
 virtual PNRStatusResponse queryPNR(const std::string& pnr) const = 0;
};
}
...

```

## Component 2: InventoryEngine Interface (`IInventoryEngine.hpp`)

```

...cpp
namespace RRS::Inventory {
 struct AvailabilityQuery {
 uint32_t trainID;
 std::string journeyDate;
 CoachClass travelClass;
 QuotaType quota;
 };

 struct AvailabilityResult {
 uint16_t confirmedAvailable;
 uint16_t racAvailable;
uint16_t waitlistCount;
 BookingStatus currentStatus; // AVAILABLE, RAC, WL, REGRET
 };

 class IInventoryEngine {
 public:
 virtual ~IInventoryEngine() = default;
 virtual AvailabilityResult checkAvailability(const AvailabilityQuery& query) const = 0;
 virtual std::vector<AllocatedBerth> allocateBerths(uint32_t trainID, const std::string& date,
CoachClass travelClass, const std::vector<PassengerInput>& passengers) = 0;
 virtual CascadeResult executeWaitlistCascade(uint32_t trainID, const std::string& date,
CoachClass travelClass, uint16_t vacatedBerthCount) = 0;
 };
}
...

```

## 4.4 Error Handling, Logging & Monitoring

- Standardized Error Codes:** All module interfaces return strongly-typed enum status codes ('StatusCode::SUCCESS', 'ERR\_AUTH\_FAILED', 'ERR\_SEATS\_FULL', 'ERR\_INVALID\_PNR', 'ERR\_LOCK\_TIMEOUT'). Exceptions are isolated to internal I/O layers.
- Masked Logging Policy:** Credentials, passwords, and passenger identities are strictly excluded from debug traces. Production logs record event codes, timestamps, and masked identifiers.
- Real-Time Monitoring Hooks:** Runtime health monitors record active thread counts, booking latency (target ≤ 500 ms), mutex contention wait times, and daily transaction throughput.

## 4.5 UX Design

- The Terminal User Interface (TUI) adheres to ANSI VT100 console standards, designed for rapid keyboard navigation:
- High-contrast visual hierarchy: Navy headers (`\033[1;34m``), success prompts (`\033[1;32m``), and critical alerts (`\033[1;31m``).
- Single-keystroke numeric accelerators for counter clerks (e.g. ``[1`` Search, ``[2`` Book, ``[3`` Cancel, ``[4`` PNR Status).
- Aligned ASCII tabular grids for train schedules, berth seat matrices, and booking summaries.
- Accessible text-based fallback ensuring 100% compatibility across legacy station terminals and modern SSH shells.

## 4.6 Open Issues & Next Steps

- **Advanced Demand-Pricing Engine:** Dynamic pricing algorithm utilizing machine learning for seasonal festival demand surges.
- **Biometric Hardware Authentication:** Biometric fingerprint verification for booking clerk authentication at physical counters.
- **Multi-Node Database Replication:** Distributed Raft consensus protocol for multi-station high-availability database replication.

# 5. Appendices

## 5.1 Glossary

Definitions of technical terms: RRS, PNR, CNF, RAC, WL, FIFO, ADR, STRIDE, RBAC, ACID, RAII, POSIX.

## 5.2 Architectural Decision Records (ADRs)

- ADR-001: Layered Architecture Selection  
Context: Need high throughput and low latency on embedded workstation terminals.  
Decision: Adopt Layered Architecture with in-process C++ components. Microservices rejected due to network and serialization overhead.  
Consequences: High performance ( $<150$  ms query latency); requires strict in-process concurrency management via mutexes.
- ADR-002: Fixed-Length Binary Record Storage with Write-Ahead Logging (WAL)  
Context: System must run autonomously without requiring external heavy database servers (Oracle, MySQL). Decision: Implement custom binary file storage (`.dat`) with fixed-stride records and write-ahead logging.  
Consequences: Zero external dependencies; microsecond  $O(1)$  disk seeks via `fseek`; guarantees crash resilience.
- ADR-003: Concurrency Control via POSIX Mutexes and Record Locks (`fcntl`)  
Context: Multi-clerk counters can simultaneously book the final available berth.  
Decision: Enforce fine-grained per-train in-memory mutexes for memory structures and exclusive record locks for disk commits.  
Consequence: Completely eliminates double-booking race conditions while allowing concurrent bookings on separate trains.
- ADR-004 : Anti-Fraud 32-Bit CRC Checksum in 10-Digit PNRs  
Context: Prevent manual fabrication or alteration of ticket identifiers.  
Decision: Embed a cyclic redundancy checksum within 10-digit PNR structure.  
Consequence: Instant offline validation of forged PNRs without needing full database lookups.

### 5.3 References

1. IEEE Std 42010-2011: Systems and software engineering — Architecture description.
2. ISO/IEC 25010:2011: Systems and software Quality Requirements and Evaluation (SQuaRE).
3. OWASP Top 10 Security Architecture & Design Principles.
4. NIST SP 800-160: Systems Security Engineering.

### 5.4 Tools and Environments

PlantUML, Matplotlib, GCC 11.4+, Clang 14.0+, Valgrind v3.18+, LLVM AddressSanitizer, GDB.